BiblioKeep Frontend - Arquitectura Profesional Completada ☑

📊 Resumen del Proyecto

Stack Implementado

- **Angular 21** - Standalone Components
- **TypeScript 5.9** - Strict type checking
- **Tailwind CSS 4.1** - Utility-first CSS
- **Lucide Angular** - Iconos profesionales
- **RxJS 7.8** - Reactive programming
- **Signals** - State management

🏗 Arquitectura Final

Estructura de Carpetas

```
src/app/
│
├── 📦 core/                          # Lógica central
│   ├── components/                   # Componentes core (layouts)
│   │   ├── main-layout/              # Layout principal
│   │   │   ├── main-layout.component.ts
│   │   │   ├── main-layout.component.html
│   │   │   ├── main-layout.component.css
│   │   │   └── GUIDE.md
│   │   └── index.ts
│   ├── models/                       # Interfaces TypeScript
│   │   ├── user.ts                   # UUID id, email, preferences, goal
│   │   ├── book.ts                   # Book entity + BookStatus enum
│   │   ├── loan.ts                   # Loan entity
│   │   ├── auth-response.ts          # JWT + token + user
│   │   └── index.ts
│   ├── services/                     # Servicios HTTP
│   │   ├── auth.service.ts           # ☑ Login, Register, Logout
│   │   ├── book.service.ts           # ☑ CRUD + Búsqueda híbrida
│   │   ├── loan.service.ts           # ☑ CRUD + Préstamos vencidos
│   │   ├── stats.service.ts          # ☑ Dashboard stats
│   │   └── index.ts
│   └── interceptors/                 # Interceptores HTTP
│       ├── auth.interceptor.ts       # ☑ Inyecta JWT automáticamente
│       └── index.ts
│
├── 📦 shared/                        # Componentes reutilizables
│   ├── components/                   # Componentes atómicos
│   │   ├── button/                   # ☑ 3 variantes (primary, secondary,
```

```
danger)
│   │   ├── book-card/                # ☑ Card de libro con rating
│   │   ├── stats-widget/             # ☑ Widget de estadísticas
│   │   ├── form-input/               # ☑ Input reutilizable
│   │   └── index.ts
│   └── utils/                        # Utilidades compartidas (future)
│
├── 📦 features/                       # Módulos de features
│   ├── dashboard/                    # ☑ Estadísticas y métricas
│   │   ├── dashboard.component.ts    # Carga stats en tiempo real
│   │   └── pages/
│   ├── library/                      # ☑ Gestión de libros
│   │   ├── library.component.ts      # Grid de libros + CRUD
│   │   └── pages/
│   └── loans/                        # ☑ Gestión de préstamos
│       ├── loans.component.ts        # Tabla de préstamos
│       └── pages/
│
├── app.ts                            # Root component
├── app.routes.ts                     # Configuración de rutas
├── app.config.ts                     # Configuración global (HTTP, providers)
├── app.html                          # Root template
└── app.css                           # Estilos globales
```

🔧 Servicios Implementados

AuthService ☑

```
- login(email, password): Observable<AuthResponse>
- register(email, password): Observable<AuthResponse>
- logout(): void
- getToken(): string | null
- getCurrentUser(): User | null
- isAuthenticated(): boolean
- user$: Observable<User | null>
- token$: Observable<string | null>
```

BookService ☑

```
- getBooks(): Observable<Book[]>
- getBook(id): Observable<Book>
- createBook(book): Observable<Book>
- updateBook(id, book): Observable<Book>
- deleteBook(id): Observable<void>
- updateBookStatus(id, status): Observable<Book>
- searchBooks(query): Observable<Book[]>          // Búsqueda híbrida
- searchByIsbn(isbn): Observable<Book>            // Por ISBN
```

LoanService ☑

```
- getLoans(): Observable<Loan[]>
- getLoan(id): Observable<Loan>
- createLoan(loan): Observable<Loan>
- updateLoan(id, loan): Observable<Loan>
- markAsReturned(id): Observable<Loan>
- deleteLoan(id): Observable<void>
- getOverdueLoans(): Observable<Loan[]>
```

StatsService ☑

```
- getDashboardStats(): Observable<DashboardStats>
```

🔐 Seguridad

AuthInterceptor

- ☑ Inyecta `Authorization: Bearer <token>` automáticamente
- ☑ Se aplica a todas las peticiones HTTP
- ☑ Configurable en `app.config.ts`

Estado de Autenticación

- ☑ Persistencia en `localStorage`
- ☑ Recuperación al cargar la app
- ☑ Observables reactivos para estado compartido

🎨 Componentes Atómicos

ButtonComponent

```
<app-button
  label="Guardar"
  variant="primary" | "secondary" | "danger"
  [isDisabled]="false"
  (onClick)="handler()"
></app-button>
```

BookCardComponent

```
<app-book-card [book]="book"></app-book-card>
```

StatsWidgetComponent

```
<app-stats-widget
  [stat]="{ label, value, icon, color }"
></app-stats-widget>
```

FormInputComponent

```
<app-form-input
  label="Email"
  type="email"
  placeholder="user@example.com"
  (onValueChange)="handler($event)"
></app-form-input>
```

📱 Features Implementados

Dashboard ☑

- Grid responsivo de widgets (1, 2, 4 columnas)
- Barra de progreso de meta anual
- Carga de estadísticas en tiempo real
- Cards con gradientes y colores

Mi Biblioteca ☑

- Grid de libros con portada y rating
- Estados de lectura con badges
- Botón para agregar libros
- Estado vacío personalizado

Préstamos ☑

- Tabla responsiva de préstamos
- Estados (Activo/Devuelto)
- Acciones por fila
- Marcar como devuelto in-place

🎯 Características Principales

Característica	Estado	Notas
Estructura profesional	☑	Core/Shared/Features

Característica	Estado	Notas
Interfaces TypeScript	☑	Alineado con backend
Servicios HTTP	☑	CRUD completo
Autenticación JWT	☑	Token + localStorage
Interceptor HTTP	☑	Automático
Componentes atómicos	☑	Reutilizables
Tailwind CSS	☑	100% puro
Lucide Angular	☑	Iconos dinámicos
Signals	☑	Estado reactivo
OnPush Detection	☑	Rendimiento optimizado
Responsive Design	☑	Mobile-first
Sidebar collapsable	☑	Drawer en móvil
Dark mode ready	🔄	Preparado

🚀 Próximos Pasos

Fase 4: Protección de Rutas

- ☐ AuthGuard (verificar token)
- ☐ RoleGuard (verificar permisos)
- ☐ Redirección a login

Fase 5: Autenticación Completa

- ☐ Página de Login
- ☐ Página de Register
- ☐ Recuperación de contraseña
- ☐ Validación de formularios

Fase 6: Modales y Formularios

- ☐ Modal para agregar libros
- ☐ Modal para crear préstamos
- ☐ Validación reactiva
- ☐ Manejo de errores

Fase 7: Mejoras UX

- ☐ Toast notifications
- ☐ Loading spinners
- ☐ Confirmación de acciones
- ☐ Mensajes de error descriptivos

Fase 8: Testing

- ☐ Tests unitarios (servicios)
- ☐ Tests de componentes
- ☐ E2E tests (Cypress/Playwright)

📋 Checklist Final

- ☑ Estructura de carpetas profesional
- ☑ Interfaces TypeScript alineadas con backend
- ☑ Servicios HTTP (Auth, Book, Loan, Stats)
- ☑ Interceptor de autenticación
- ☑ Componentes atómicos reutilizables
- ☑ MainLayout responsivo
- ☑ Features con lógica
- ☑ Signals para estado
- ☑ OnPush change detection
- ☑ Tailwind CSS 100%
- ☑ Lucide Angular integrado
- ☑ Compilación sin errores
- ☑ Hot reload activo

🎓 Principios Aplicados

- **DRY** (Don't Repeat Yourself) - Componentes reutilizables
- **SOLID** - Servicios con responsabilidad única
- **Scalability** - Estructura que crece sin problemas
- **Accessibility** - ARIA labels y navegación por teclado
- **Performance** - OnPush CD + Signals
- **Type Safety** - TypeScript strict mode

Proyecto: BiblioKeep Frontend
Estado: 60% - Marco + Servicios + Componentes
Próximo Hito: AuthGuard + Login/Register
Última Actualización: 2026-01-29
Contacto/Repo: GitHub DevSenior Módulo 9